- **业务场景**
  - 需求描述
    - 项目由多个独立的部分组成，同时每个独立的部分需要单独部署
    - 项目的一部分是之前已经存在了的项目
  - 示例
    - 父项目
      - 子项目：强对流预警
      - 子项目：雷达单站
      - 子项目：雷达三维
      - 子项目：评估检验
      - 子项目：个例复盘

- **解决方案**
  - iframe
    - 父子通信
      - 本地存储（要求同源）
        - 示例：利用相同的key存取token
      - 全局数据（要求同源）
        - 说明：父项目给自己的window添加属性，子项目读取使用
        - 示例代码
          - 父项目

```
window.globalData = {
  headerHeight,
  tokenKey,
  parentRouter: router,
  loginPath,
  redirectKey,
  ElMessage
}
```

          - 子项目

```
export let iframeGlobalData: Window['globalData']
try {
  iframeGlobalData = window.top?.globalData
} catch (error) {
```

```
  console.error(error)
}
```

- 类型代码（创建.d.ts类型声明文件，引入到tsconfig.json中）

```
/// <reference types="vue-router" />

interface Window {
  globalData?: {
    headerHeight: number
    tokenKey: string
    parentRouter: import('vue-router').Router
    loginPath: string
    redirectKey: string
    ElMessage: import('element-plus/es/utils').SFCInstallWithContext<import('element-plus').Message>
  }
}
```

  - 事件
    - 参考：window.postMessage
- 子项目页面标题高度
  - 说明
    - 单独部署时使用自己的页面标题高度
    - 父子结构中，使用父项目统一的页面标题高度，以保证父项目的页面标题可以完美覆盖子项目的页面标题
  - 示例代码

```
const headerHeight = `${iframeGlobalData?.headerHeight ?? 60}px`
```

- 子项目登录系统
  - 说明
    - 单独部署时有自己的登录系统
    - 父子结构中，使用父项目的登录系统
  - token
    - 处理方式
      - 单独部署时，使用自己的key存取token

        - 父子结构中，使用父项目统一的key存取token
    - 示例代码

```
export const tokenKey = iframeGlobalData?.tokenKey ?? 'token-tjgz-warning'
```

- 跳转到登录页
    - 处理方式
        - 单独部署时，需跳转到登录页面时，跳转到自己的登录页面
        - 父子结构中，需要跳转到登录页面时，使父项目跳转到登录页
    - 示例代码
        - 路由守卫

```
router.beforeEach(to => {
  if (getToken()) {
    if (to.path === loginPath) {
      return '/'
    } else {
      return true
    }
  } else {
    if (to.path === loginPath) {
      return true
    } else {
      if (iframeGlobalData) {
        const parentRouter = iframeGlobalData.parentRouter
        const parentRoute = parentRouter.currentRoute.value
        parentRouter.push({
          path: loginPath,
          query: parentRoute?.path === '/' ? {} : { [redirectKey]: parentRoute?.path }
        })
        return true
      } else {
        return {
          path: loginPath,
          query: to.path === '/' ? {} : { [redirectKey]: to.path }
        }
      }
    }
```

```
  }
})
```

- 请求拦截器

```
instance.interceptors.response.use(
  response => {
    if (+response.data.code === 200 || (response.status === 200 && response.data instanceof Blob)) {
      return response.data
    } else {
      const rawRouter = iframeGlobalData?.parentRouter ?? router
      const currentPath = rawRouter.currentRoute.value.path
      if (+response.data.code === 401 && currentPath !== loginPath) {
        removeToken()
        rawRouter.push({
          path: loginPath,
          query: currentPath === '/' ? {} : { [redirectKey]: currentPath }
        })
      }
      return Promise.reject(response.data)
    }
  },
  error => Promise.reject(error)
)
```

- ElMessage
  - 说明
    - 子项目的ElMessage会被父项目的页面标题挡住，子项目中使用ElMessage时需要使用父项目的ElMessage
  - 示例代码（子项目）

```
export const getElMessage = () => iframeGlobalData?.ElMessage ?? ElMessage
```

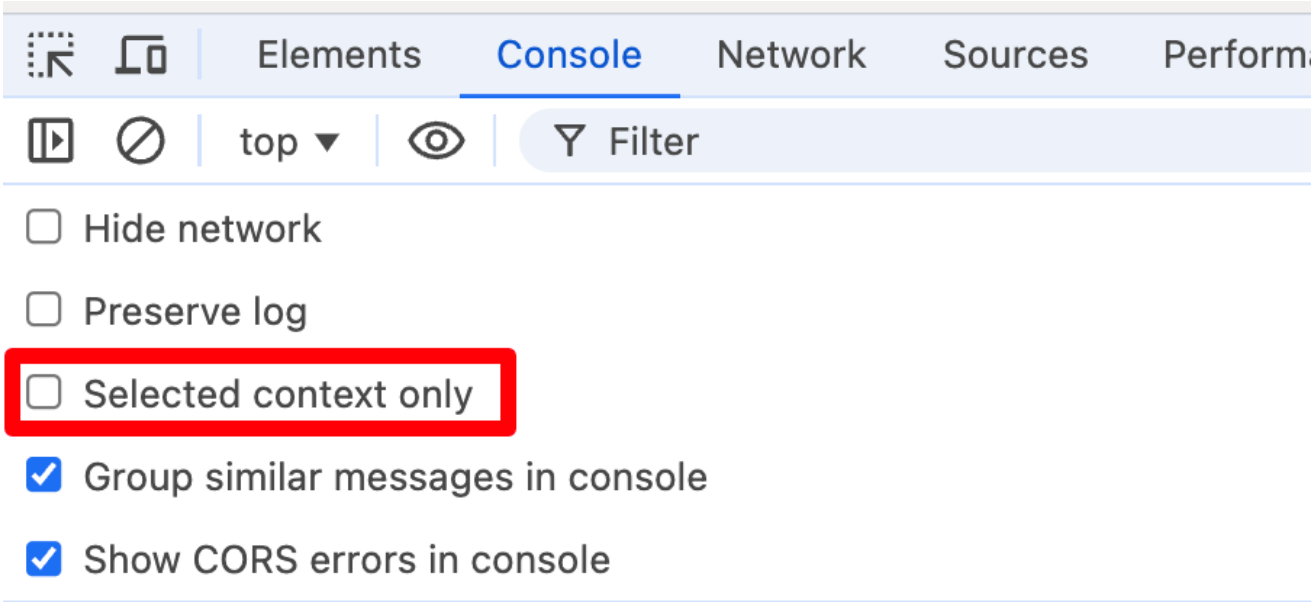
- 父子项目独立部署，且保证同源

```
## 父项目
```

```
location /tjgz/ {
    alias /home/bgdb/nowcastings/tjgz/;
    try_files $uri $uri/ /tjgz/index.html;
}

## 子项目
location /tjgz-warning/ {
    alias /home/bgdb/nowcastings/tjgz-warning/;
    try_files $uri $uri/ /tjgz-warning/index.html;
    add_header Cache-Control no-cache;
}
```

- 子项目设置协商缓存验证
  - 发现问题
    - 子项目重新部署后，单独访问子项目时页面自动更新，但从父项目的iframe里访问子页面时不会自动更新，必须手动清除缓存后才可以更新。
  - 问题产生原因
    - 浏览器加载网页时，响应头返回了Etag、Last-Modified字段，可用于下次请求时验证资源是否改变，并且使得浏览器缓存了页面。但未设置Cache-Control，未强制要求下次请求时验证资源。
    - 浏览器再次直接访问项目时，请求头中携带了Cache-Control：max-age=0（要求验证资源是否改变），使得当次请求携带上一次的请求的Etag、Last-Modified字段进行资源验证，页面未改变就走缓存（304状态码），改变了就重新获取。
    - 在iframe访问项目时，请求头未携带Cache-Control字段，未要求此次请求验证资源，而是直接使用强制缓存访问缓存内容，未能获取更新。
    - 综上，响应头未明确指定缓存方式，而浏览器和iframe的默认原则存在差异（浏览器默认协商缓存，iframe默认强制缓存），导致了浏览器直接访问可以及时获取更新，但在iframe中访问无法及时获取更新。
  - 解决方案
    - 在nginx里配置子项目的响应头中添加Cache-Control: no-cache字段，明确指定缓存策略为协商缓存。
- 查看子项目的控制台打印
  - 取消勾选浏览器控制台Console面板的Selected context only



- 微前端框架
  - 概念
    - 微前端借鉴了微服务的架构理念，核心在于将一个庞大的前端应用拆分成多个独立灵活的小型应用，每个应用都可以独立开发、独立运行、独立部署，再将这些小型应用融合为一个完整的应用，或者将原本运行已久、没有关联的几个应用融合为一个应用。
  - iframe的缺陷
    - 父子项目非同源部署时通信不便
    - 父子项目url不同步，且刷新后子项目的url状态丢失
    - DOM结构不共享，原生硬隔离无法突破
    - 慢，每次加载子应用都要重建浏览器上下文
  - 主流框架
    - single-page
      - 上手难度最大，不建议使用
    - qiankun
      - 上手难度较大，且与vite不兼容，不建议使用
    - micro-app（最建议使用）
      - 上手简单，建议使用
    - wujie
      - 上手简单，建议使用